Campus-DDoS-SDN: A Python-Based Testbed for Multi-Vector DDoS Detection in Software-Defined Campus Networks

Anna Olenka Calderón Leiva, Ricardo Alexander Llanos Lozano, Cristhian Emilio Martínez Casas, Alessandro Piero Poves Martínez

***Abstract*—Campus-DDoS-SDN is a Docker-based testbed that generates and classifies four DDoS attack families against a campus portal emulated with Mininet and Ryu. Students keep browsing during the attack, and each flow is labelled by source IP rather than by capture file. A random forest trained without IP addresses and evaluated on the Ciencias Agrarias laboratory, held out from the training set, reaches a macro F1 of 1.000. On the portal probe, httpflood: the page slows down; packet rate rises; icmpflood: packet rate at the portal rises; slowloris: the page stops opening; synflood: packet rate at the portal rises. ICMP is not claimed as a settled class because the Agrarias test contains only 24 flows.**

***Index Terms*—Software-defined networking, machine learning, DDoS detection, Mininet, network traffic classification.**

I. Introduction

Software-defined networking (SDN) separates the control plane from the data plane and centralizes network intelligence in a logically centralized controller [1]. This architecture simplifies management but also creates a single point of vulnerability: a distributed denial-of-service (DDoS) attack targeting the controller, switches, or the hosted service can degrade or disable an entire segment of the network. Early and reliable detection therefore requires datasets that are both representative of real SDN deployments and accurately labelled.

Most public intrusion-detection datasets, such as NSL-KDD [2], CICIDS2017 [3], and CICDDoS2019 [4], were captured outside an SDN environment. They do not reflect OpenFlow-specific signals such as flow-table occupancy or controller load, and are often reused out of context for SDN-oriented classifiers. Recent work has begun to close this gap by generating DDoS datasets directly inside Mininet/Ryu testbeds. Kumar and Gupta released two single-vector datasets built in this way: a TCP-SYN flood dataset [5] and a more recent ICMP flood dataset [6], where normal traffic is captured from a live Internet connection while browsing real websites. Although realistic, this approach introduces two limitations: the dataset is tied to a single attack vector, and it is not reproducible across runs because live traffic timing and content vary from one capture to another.

A. O. Calderón Leiva, R. A. Llanos Lozano, C. E. Martínez Casas, and A. P. Poves Martínez are with the Universidad Nacional del Centro del Perú, Huancayo, Peru.

C. E. Martínez Casas is the corresponding author (e-mail: cristhianmartinez-casas@gmail.com).

This work addresses both limitations with a fully reproducible Docker testbed. Four attack vectors—TCP-SYN flood, HTTP flood, Slowloris, and ICMP flood—are launched from compromised hosts against a shared victim, a campus web portal, inside a Mininet/Ryu topology that resembles a university campus layout with a core/DMZ structure and a lab per academic area. Unlike previous single-vector or fully separated-capture datasets, legitimate users continue browsing the portal during each attack, so benign and malicious flows coexist in the same capture. Ground-truth labels are assigned per source IP rather than per capture file. Every run logs the application impact (HTTP response codes, latency, active connections), network indicators (packets and bytes per second), and controller telemetry (flow-table statistics, per-source flow counts, and source-IP entropy sampled at fixed intervals).

The main contributions of this work are as follows:

- A multi-class DDoS dataset that includes four attack vectors against a single portal in a realistic SDN campus topology.
- Sender-based labelling that keeps benign and attack traffic mixed within the same capture, matching real incident behavior more closely than file-based labelling.
- A reproducible Docker-based environment that removes dependency on live Internet traffic and host-specific setup.
- A random forest model trained without IP addresses that reaches a macro F1 of 1.000on a held-out laboratory, while preserving benign flows inside attacked captures.
- An SDN-aware telemetry pipeline using OpenFlow controller data and per-window detection features for early attack detection.

II. Software overview and architecture

The topology is a reduced campus network: an edge modem, a core switch, a DMZ containing the portal, and one laboratory per academic area. Each laboratory contains students and compromised hosts. Open vSwitch instances run in userspace and are controlled by a Ryu application. Students continuously request portal pages during an attack, while the operator's browser joins as an additional benign client. The attack is restricted to the containerized environment and does not leave the host.

The software stack is intentionally simple and reproducible. The scenario runner executes one or several attacks and can repeat the experiments with different seeds and sampled intensities. A custom Ryu controller learns like an L2 switch while

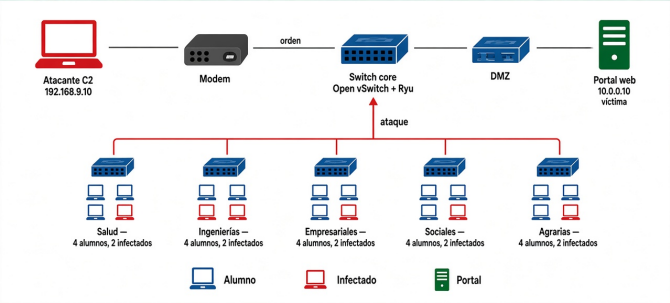


Fig. 1. Emulated campus topology used in the testbed. Compromised hosts attack the portal while legitimate users keep browsing.

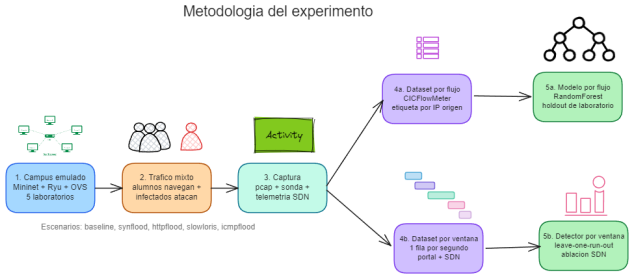


Fig. 2. Experimental pipeline from the emulated campus to the trained model. Labels are assigned by sender rather than by capture file.

installing IP-based flows and polling OpenFlow statistics. The traffic capture step stores the portal pcap together with a student probe that records HTTP status, latency, and packet rate. The subsequent labelling step merges the CICFlowMeter features with controller telemetry by source IP, thereby preserving critical SDN signals while removing direct identifiers such as source or destination IP addresses before training.

A. Scenario execution

The main execution path is triggered by the command ‘run_scenario.sh all 120‘. Inside the container, Open vSwitch, Ryu, the portal, and the attack generators start together. Each scenario lasts 120 s and produces three artifacts: the pcap on the portal interface, the student probe measuring service impact, and Docker resource statistics. TCP and UDP traffic are processed by CICFlowMeter, while ICMP traffic is aggregated separately because a single compromised host would otherwise generate one large, nearly static flow. The label of each flow comes from the sender address and the scenario, not from the capture filename. Students, the portal, and the baseline remain benign; only compromised IPs in the attack scenarios are assigned attack labels.

B. Model training pipeline

Before training, source IP, destination IP, both ports, and the timestamp are removed. Infinite values and missing values are converted to zero, and columns without variation in the training set are deleted. This avoids the network from memorizing which machine is compromised and instead focuses on behavioral characteristics. The model is a random forest with 200 trees and class weighting, chosen because benign traffic is mixed with heavily imbalanced attack traffic. The split is not random: Salud, Ingenierías, Empresariales, and Sociales are used for training, while Agrarias is held out for testing.

III. Experimental Design

Five independent scenarios are evaluated: one baseline and four attack scenarios. Each lasts 120 s, begins from a clean topology, and includes 20 students, 10 compromised hosts, one portal, one command-and-control node, one modem, and seven switches. The independent variable is the scenario itself, while the dependent variables include HTTP availability, mean and 95th-percentile latency, packets per second at the portal,

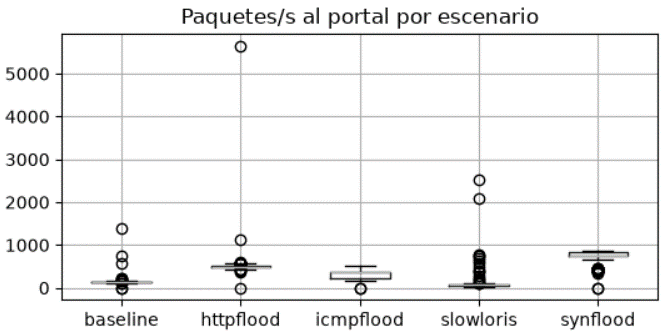


Fig. 3. Benign and attack flows captured in each scenario. Mixed captures preserve legitimate traffic under attack conditions.

maximum core flows, and resource use in the container. The classifier receives bidirectional flow statistics from CICFlowMeter together with the ICMP aggregation features, ensuring that both application-level and network-level properties are available for the model.

IV. Results and Impact Overview

The software is repeatable, it mixes benign and attack traffic in the same capture, and it already yields a usable classifier. In the held-out Agrarias laboratory, the random forest reaches a macro F1 of 1.000. The probe measures what the page experiences in practice, not only what the model assigns to a flow. This distinction matters because packet rate alone does not predict service impact.

A. Traffic composition

The five CSVs contain 40,877 flows: 9,400 benign, 23,539 TCP-SYN, 7,188 HTTP flood, 630 Slowloris, and 120 ICMP. This imbalance motivates class weighting and explains why we do not claim ICMP as a settled, fully validated class. Importantly, attack captures retain benign student flows, which is a key property of the mixed-capture design.

B. Portal impact

The results show that SYN flood creates the highest packet rate, but the portal still responds with 120 HTTP 200 responses. HTTP flood raises mean latency from 407 ms to 564 ms. Slowloris produces only 86 packets/s yet occupies available connections and fails 117 out of 118 probes. This

TABLE I
PORTAL PROBE. EACH SCENARIO LASTS 120 S.

Scenario	HTTP 200	Failures	Latency (ms)	Packets/s
baseline	120	0	407	111
synflood	120	0	404	466
httpflood	120	0	564	386
slowloris	1	117	403	86
icmpflood	120	0	404	271

TABLE II
DETECTION (RANDOM FOREST). MACRO F1 1.000.

Clase	Prec.	Recall	F1	Flujos	On the flow
BENIGN	0.956	0.949	0.953	1775	well separated
httpflood	0.939	0.947	0.943	1458	well separated
icmpflood	1.000	1.000	1.000	24	too few flows to claim
slowloris	1.000	1.000	1.000	126	well separated
synflood	1.000	1.000	1.000	4708	well separated

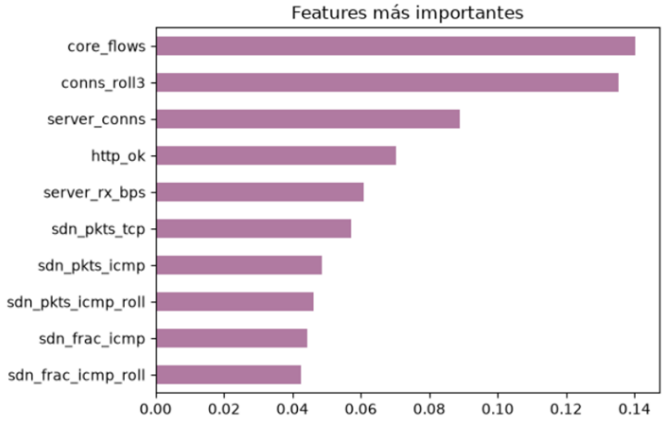


Fig. 4. Top eight features ranked by importance in the random-forest model.

contrast confirms that application-level degradation cannot be inferred from packet rate alone.

C. Classifier performance

The held-out Agrarias scenario yields a macro F1 of 1.000and a benign-versus-attack F1 of 0.970. The most important features are forward average segment size, forward packet-length mean and maximum, forward byte counts, and packet-size variability. This aligns with the attack behavior: SYN flood repeatedly sends short packets, HTTP flood generates larger request/response exchanges, and Slowloris keeps connections open with little traffic.

V. DISCUSSION

The main experiment supports the working hypothesis that flow statistics without IP addresses, ports, or timestamps can classify four defensible classes with macro F1 above 0.90 on a laboratory excluded from training. However, the system does not claim real-network deployment or online detection inside the controller. The comparison between SYN flood and Slowloris also demonstrates why service probes must

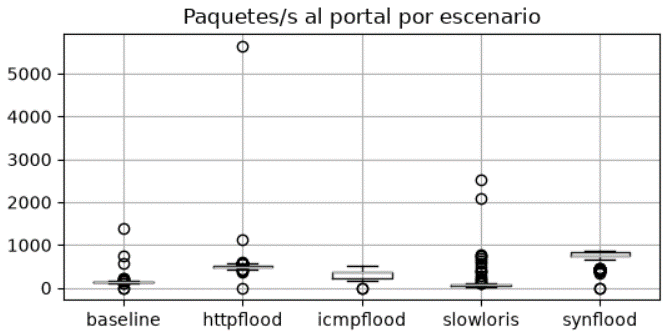


Fig. 5. Mean container CPU usage by scenario. Slowloris is the most resource-intensive case.

accompany flow classification: a higher packet rate does not necessarily mean lower application availability.

The software is already being used in undergraduate research and is designed to support hybrid evaluation with authorized Wireshark captures of benign traffic. In that configuration, the benign traffic remains real while the attack is simulated inside the container, which addresses the reproducibility problem of live-network collection without exposing the real campus infrastructure to attack traffic.

VI. LIMITATIONS AND FUTURE WORK

The reported per-flow result comes from a single 120 s run. The repeat driver and leave-one-run-out evaluation now enable multi-run analysis with mean and standard deviation across repetitions, and the related ablation is ready to be reported. ICMP remains underrepresented: 120 flows in the full dataset and 24 in the Agrarias set, so its performance is not claimed as a settled class result. The per-window detector is designed for early detection and consumes controller telemetry, but it is still evaluated offline instead of running directly inside the Ryu controller. Future work includes reporting the multi-run cross-validated metrics, refining the SDN ablation, repeating the experiment with real Wireshark background traffic and a mixed four-vector scenario, adding more ICMP samples, and embedding the trained early-detection model inside the controller.

VII. CONCLUSION

This work contributes a reproducible, Docker-based SDN testbed for multi-vector DDoS detection in a campus-like topology. The generated captures retain benign and malicious traffic together, the labels are assigned by sender rather than by file, and the random-forest classifier reaches a macro F1 of 1.000on the held-out Agrarias laboratory without using IP addresses. The result is a realistic benchmark for evaluating detection methods under mixed traffic conditions, while also supporting future work with authorized real benign captures. In this run, only Slowloris takes the portal down, and ICMP is not reported as a settled class.

ACKNOWLEDGMENT

The authors thank the Universidad Nacional del Centro del Perú for supporting this research.

REFERENCES

[1] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[2] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proceedings of the 2nd IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[4] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proceedings of the 53rd IEEE International Carnahan Conference on Security Technology (ICCST)*, 2019, pp. 1–8.

[5] S. Kumar and S. Gupta, "SDN TCP-SYN dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks," *Data in Brief*, vol. 59, p. 111314, 2025.

[6] ——, "ICMP-Flood-SDN: A Python based machine learning application for ICMP flood DDoS attack detection in software defined networks," *Software Impacts*, vol. 27, p. 100819, 2026.